Команда GoatWhistle

ИИ-АССИСТЕНТ ПОСТРОЕНИЯ BPMN-СХЕМ SCHEMIX

Материалы ИТ-проекта

Описание предметной области, модель данных, модель функционирования,
процессная модель, архитектура, технический проект ПО,
требования к программному компоненту, оценка качества

2026

СОДЕРЖАНИЕ

1 Описание предметной области 3
2 Модель данных 7
3 Модель функционирования 12
4 Процессная модель 16
5 Архитектура 21
6 Технический проект программного обеспечения 24
7 Требования к программному компоненту «Агент» 27
8 Оценка качества 33

1 Описание предметной области

1.1 Объект автоматизации

В энергокомпании порядок работы закрепляют регламенты: кто выполняет шаг, в какой последовательности, при каком условии работа уходит на согласование. Регламент пишут текстом, а схему процесса в нотации BPMN 2.0 аналитик рисует вручную в графическом редакторе. Текст и схема быстро расходятся: при каждой правке регламента схему перерисовывают, на это уходит время, и ошибки в условиях и ветвлениях замечают поздно.

schemix — ИИ-ассистент, который строит схему BPMN 2.0 по описанию процесса и даёт аналитику править её. Описание можно задать текстом в чате, файлом (.docx, .pdf, .txt, .md) или голосом. Готовую схему можно открыть в bpmn.io или Camunda Modeler.

1.2 Участники

– **Аналитик** (бизнес-аналитик, методолог) — единственный пользователь системы. Описывает процесс, отвечает на уточняющие вопросы, подтверждает или отклоняет изменения, правит схему вручную, выгружает результат. Входа в систему и разграничения прав нет: у всех пользователей стенда одно общее рабочее пространство.

– **Провайдер языковой модели** — внешний сервис с OpenAI-совместимым интерфейсом chat/completions (DeepSeek V4.1 Flash, Qwen3.6). Получает текст и JSON-схему ответа, возвращает факты о процессе.

– **Сервис распознавания речи** — внешний сервис с интерфейсом /audio/transcriptions. Получает фразу диктовки в формате WAV и возвращает текст.

– **BPMN-редакторы** (bpmn.io, Camunda Modeler) — получатели выгруженных файлов и источники файлов для импорта.

1.3 Основные объекты

– **Диаграмма** — схема одного процесса. У неё есть название, номер текущей ревизии и признак редактируемости. Если импортированный файл использует элементы вне профиля редактора, диаграмма открывается только для чтения.

– **Ревизия** — неизменяемая версия схемы: полный XML BPMN 2.0 с графической разметкой (DI) и источник изменения (создание, импорт, ручная правка, агент, возврат, диктовка, этапы). Возврат к старой версии не перезаписывает историю, а создаёт новую ревизию.

– **Чат и сообщение** — переписка аналитика с агентом. У диаграммы может быть несколько чатов.

– **Запрос агенту** — обработка одного сообщения. Запрос хранит базовую ревизию, от которой агент начал работу, статус, текущий этап и ревизию-результат.

– **Раунд уточнений** — вопросы агента с вариантами ответа и ответы аналитика. В одном запросе бывает не больше трёх раундов.

– **Предложение и предпросмотр** — рискованное изменение, которое ждёт решения аналитика, и схема-кандидат для его просмотра.

– **Диктовка и фраза диктовки** — голосовой ввод, разбитый на короткие фразы.

1.4 Как работает система

Аналитик пишет в чат, что нужно сделать. Сервер сохраняет сообщение и ставит запрос в очередь. Worker агента берёт запрос по аренде (lease) и прогоняет его через семь ролей языковой модели:

– **intent** — что требуется сделать;
– **entities** — участники, шаги и события;
– **entity_match** — сопоставление с тем, что уже есть на схеме;
– **flow** — порядок шагов и условия;
– **relation_match** — какие связи меняются;
– **review** — проверка результата;
– **clarify** — вопросы с вариантами ответа.

Модель возвращает только факты с цитатами из исходного текста. Схему по этим фактам собирает программный код: он формирует набор изменений (ChangeSet), применяет его к копии схемы, раскладывает элементы и проверяет результат по XSD OMG и правилам графа.

Если фактов не хватает, агент задаёт вопрос с вариантами ответа. Рискованным код считает изменение, которое удаляет элемент или связь, переподключает связь, меняет условие, тип шлюза или элемента либо переносит

элемент к другому участнику. Такое изменение показывается как предложение с предпросмотром и ждёт подтверждения. Безопасное изменение сразу записывается новой ревизией. Ход работы, черновик и новые ревизии приходят в браузер по WebSocket.

1.5 Бизнес-правила

1) У диаграммы в каждый момент не больше одного активного запроса агенту.

2) Любая запись ревизии сверяет базовую ревизию с текущей (compare-and-set). Если схема успела измениться, запись отклоняется. Ручная правка главнее агента: готовое предложение переносится на новую ревизию, если это возможно, иначе запрос помечается устаревшим.

3) Каждая ревизия проходит проверку по XSD OMG BPMN 2.0. Схема, которая её не проходит, не сохраняется.

4) Модель не пишет XML. Она возвращает факты по JSON-схеме, а XML строит детерминированный код.

5) Каждый мутирующий запрос к API несёт заголовок `Idempotency-Key`, и повтор запроса не создаёт дубликатов.

1.6 Входы и выходы

- **Входы:**
  - текст сообщения до 20 000 символов;
  - документ до 8 МиБ;
  - голос — фразы WAV 16 кГц моно до 15 с, диктовка до 15 минут;
  - файл BPMN до 5 МиБ.
- **Выходы:**
  - схема в редакторе и история ревизий;
  - вопросы и предложения агента;
  - замечания нормоконтроля;
  - файлы BPMN 2.0, SVG, Markdown и DOCX.

1.7 Границы системы

schemix не исполняет процессы и не заменяет BPMS. Схема — это документ, а не программа для движка процессов. Система не хранит учётные за-

писи и не разграничивает права. Регламенты из корпоративных хранилищ она не загружает: исходный текст приносит аналитик.

2 Модель данных

Модель данных представлена двумя диаграммами:

– **Концептуальная модель в нотации Чена** (рисунок 1) — 12 сущностей предметной области с атрибутами и связями. Служебные таблицы (поток и журнал событий, ключи идемпотентности) и предложения этапов на неё не вынесены.

– **Логическая модель в нотации «воронья лапка»** (рисунок 2) — все 16 таблиц базы данных PostgreSQL 16 с типами столбцов, первичными и внешними ключами и кардинальностью связей. Построена по миграциям schemix (`backend/migration/versions/0001-0011`).

На логической модели связь выходит из таблицы-родителя и входит в строку внешнего ключа потомка. Цвет шапки таблицы обозначает группу:

– документ и история;

– переписка;

– агент;

– диктовка;

– служебные таблицы.

2.1 Неочевидные связи

– **`runs` ↔ `messages` — циклическая ссылка.** Сообщение пользователя порождает запрос (`runs.user_message_id`), а ответы агента ссылаются на запрос (`messages.run_id`). Оба внешних ключа отложенные (`DEFERRABLE INITIALLY DEFERRED`), поэтому сообщение и запрос создаются одной транзакцией.

– **`runs.retry_of`** — рекурсивная связь: повторный запрос ссылается на исходный (петля у `runs`).

– **`revisions.run_id`** — необязательная ссылка на запрос, результатом которого стала ревизия. У ручной правки, импорта и возврата её нет.

– **`proposals.preview_id`** — логическая ссылка на предпросмотр без ограничения FK в базе. На диаграмме она показана пунктиром.

– **`previews.diagram_id` и `structure_suggestions.diagram_id`** дублируют путь «запрос → диаграмма». Они нужны, чтобы находить записи по диаграмме без соединения с `runs`.

– **`idempotency_keys`** не связана с другими таблицами. Она защищает от повторной обработки запроса с тем же заголовком `Idempotency-Key`.

2.2 Неочевидные атрибуты

– **Два номера ревизии у запроса.** `runs.base_revision` — ревизия, от которой агент начал работу, `runs.result_revision` — ревизия, которую он записал. Если базовая ревизия перестала быть текущей, результат не применяется, а запрос получает статус «устарел» (`stale`).

– **Один активный запрос на диаграмму.** Частичный уникальный индекс `uq_runs_one_active_per_diagram` по `diagram_id` действует для статусов `queued`, `running`, `awaiting_input` и `awaiting_confirmation`.

– **`revisions.source`** — откуда ревизия: `create`, `import`, `manual`, `agent`, `restore`, `dictation`, `structure`.

– **`revisions.original_xml`** — исходный файл импорта, сохранённый байт в байт. Если схема вне профиля редактора, он же становится содержимым ревизии, и диаграмма открывается только для чтения.

– **`diagrams.current_revision`** дублирует максимальный номер ревизии. Именно по нему работает проверка compare-and-set при записи.

– **`dictation_segments.audio`** хранится только до распознавания фразы, затем стирается.

– **`run_artifacts`** — ответы ролей модели по их JSON-схемам с версиями промпта и модели. По ним можно воспроизвести, что именно вернула модель.

– **`runs.lease_token`, `runs.lease_expires_at`** — аренда запроса воркером. Если воркер упал, аренда истекает и запрос подбирает другой.

2.3 Решения в нотации Чена

– **РЕВИЗИЯ, РАУНД УТОЧНЕНИЙ и ПРАВКА ЗАПРОСА — слабые сущности.** Их ключ составной: ключ владельца плюс частичный ключ «номер» (подчёркнут пунктиром). Связи с владельцем — идентифицирующие (двойной ромб).

– **ДИКТОВКА — слабая сущность без частичного ключа.** Её первичный ключ совпадает с ключом запроса (`dictations.run_id`), то есть у одного запроса не больше одной диктовки (связь 1:1). Отдельного идентификатора у диктовки нет, поэтому частичный ключ не нужен.

– **«длительность» диктовки — производный атрибут.** Она вычисляется как разность времени последней фразы и начала диктовки.

– **Многозначные атрибуты** («причины «только чтение»», «вопросы», «причины риска» и другие) хранятся в базе как массивы JSONB.

– **Связь «опирается на»** (запрос → базовая ревизия) в базе не закреплена внешним ключом: номер базовой ревизии хранится в `runs.base_revision` без ссылки на таблицу `revisions`.

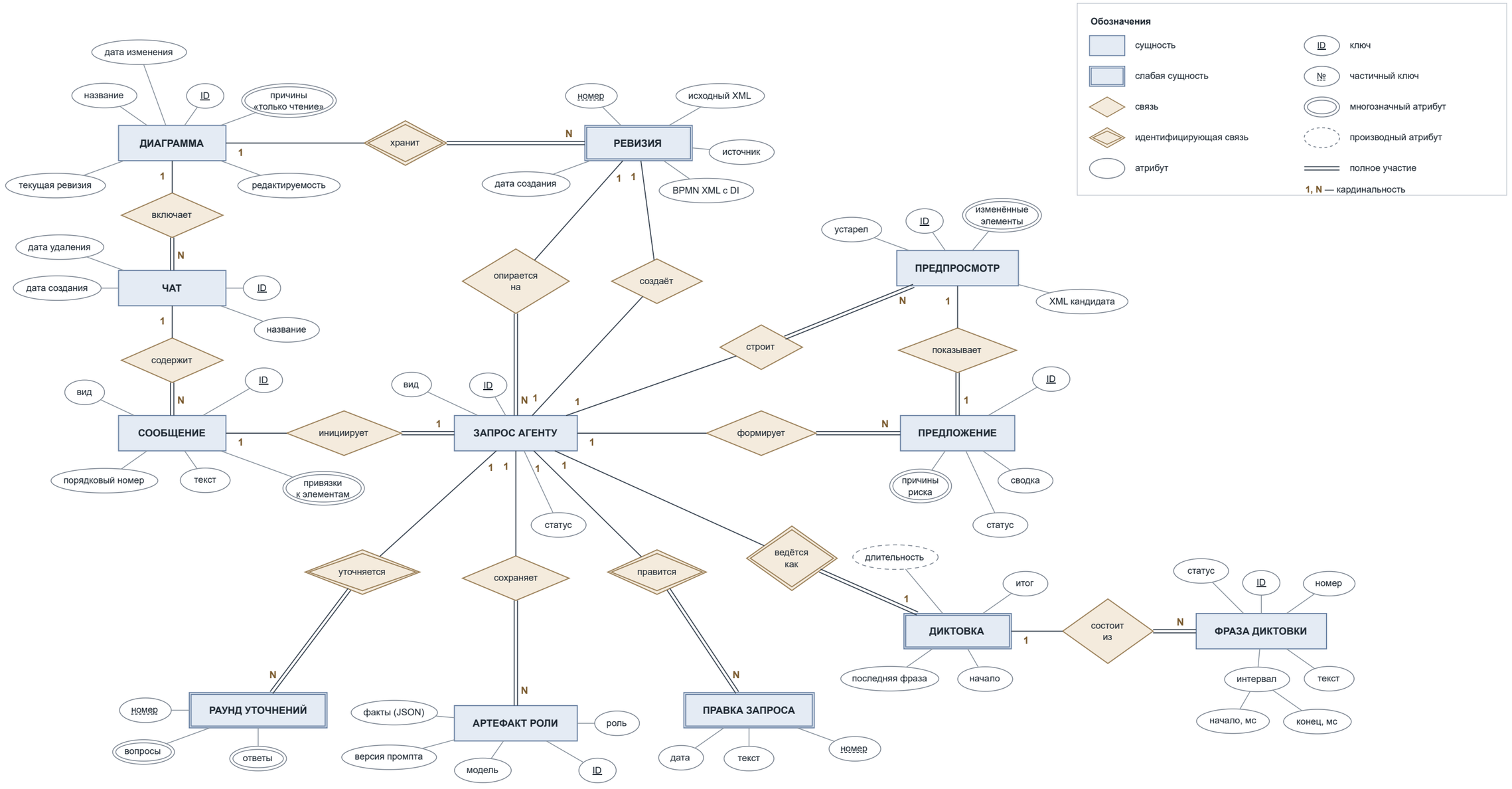


Рисунок 1 — ER-диаграмма schemix в нотации Чена (концептуальная модель данных)

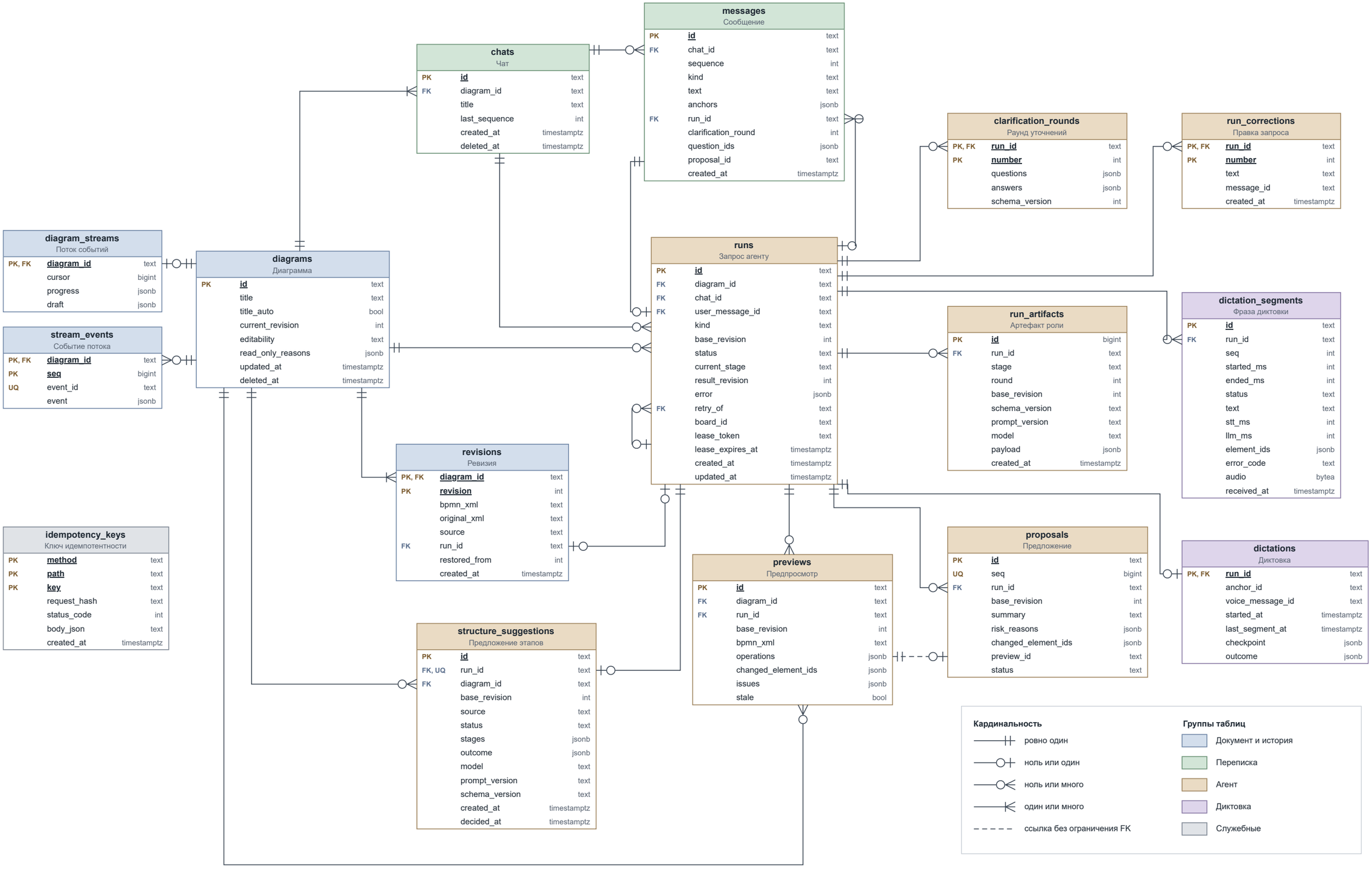


Рисунок 2 — ER-диаграмма schemix в нотации «воронья лапка» (логическая модель базы данных PostgreSQL)

3 Модель функционирования

Модель функционирования построена в виде диаграмм потоков данных (DFD) в нотации Гейна–Сарсона:

– процесс — скруглённый прямоугольник с номером в верхней полосе;

– хранилище данных — открытый справа прямоугольник с номером D;

– внешняя сущность — прямоугольник с двойной кромкой сверху и слева;

– поток данных — стрелка с подписью.

Копия внешней сущности отмечена косой чертой в углу, копия хранилища — дополнительной чертой в ячейке номера. Копии позволяют обойтись без длинных пересекающихся линий.

3.1 Контекстная диаграмма

Контекстная диаграмма (рисунок 3) показывает систему как один процесс 0 и её обмен данными с окружением:

– **с аналитиком:** описание процесса, ручные правки, ответы на вопросы, выгруженные файлы;

– **с провайдером языковой модели:** роль и JSON-схема ответа в одну сторону, факты с цитатами в другую;

– **с сервисом распознавания речи:** аудио фразы в одну сторону, её текст в другую;

– **с BPMN-редакторами:** выгруженный файл BPMN 2.0.

3.2 Диаграмма потоков данных первого уровня

На рисунке 4 процесс 0 разложен на семь процессов:

1) Принять описание процесса: текст, документ или голос.
2) Извлечь факты ролями модели.
3) Собрать изменение и проверить схему.
4) Уточнить и подтвердить изменение с аналитиком.
5) Записать ревизию.
6) Выгрузить схему и проверить оформление.
7) Транслировать события в браузер.

Хранилища D1–D5 соответствуют группам таблиц ER-диаграммы:

– D1 «Диаграммы и ревизии» — `diagrams`, `revisions`;

– D2 «Чаты и сообщения» — `chats`, `messages`;

– D3 «Запросы агенту и артефакты» — `runs`, `run_artifacts`;

– D4 «Уточнения и предложения» — `clarification_rounds`, `proposals`, `previews`;

– D5 «Журнал событий» — `diagram_streams`, `stream_events`.

Диаграмма сбалансирована с контекстной: каждый поток уровня 0 разложен на потоки уровня 1 с теми же внешними сущностями. По правилам нотации данные не переходят напрямую из хранилища во внешнюю сущность, поэтому события из D5 аналитик получает через процесс 7.

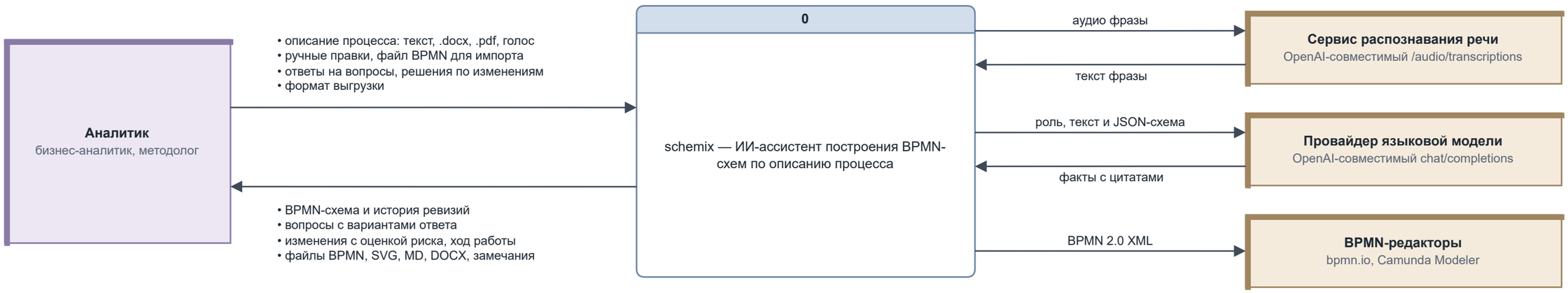


Рисунок 3 — Контекстная диаграмма schemix (DFD уровня 0, нотация Гейна–Сарсона)

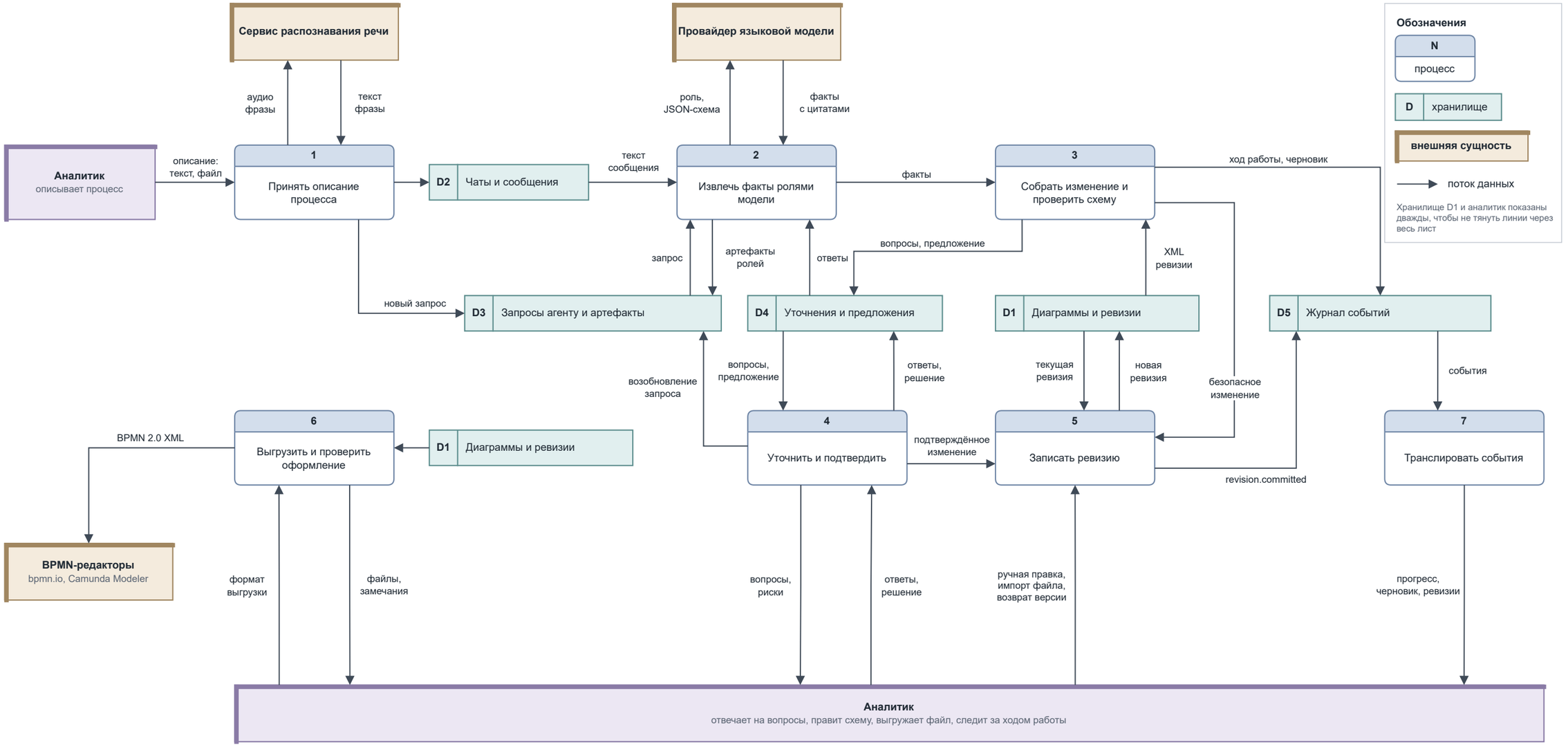


Рисунок 4 — Диаграмма потоков данных schemix (DFD уровня 1, нотация Гейна–Сарсона)

4 Процессная модель

Процессы описаны в нотации BPMN 2.0. Файлы `.bpmn` проходят проверку по XSD OMG BPMN 2.0 и открываются в bpmn.io и Camunda Modeler. Дорожки пула соответствуют исполнителям:

– аналитик;
– браузер;
– сервер schemix;
– агент.

Внешние системы показаны свёрнутыми пулами, обмен с ними — потоками сообщений. Исключающие шлюзы без подписи сливают ветви.

4.1 Построение схемы по описанию процесса

Процесс на рисунке 5 — основной сценарий системы. Аналитик описывает процесс, сервер ставит запрос в очередь, агент извлекает факты ролями модели и собирает изменение на копии схемы. Дальше возможны три исхода:

– агент задаёт вопросы, и после ответа извлечение повторяется;
– рискованное изменение уходит на подтверждение;
– безопасное изменение записывается сразу.

Перед записью сервер сверяет базовую ревизию. Если схему за это время изменили, запрос завершается как устаревший.

4.2 Ручная правка схемы

Процесс на рисунке 6 начинается с правки на холсте. Браузер автоматически отправляет XML вместе с номером базовой ревизии. Сервер блокирует диаграмму и сверяет ревизию. При конфликте аналитик выбирает: загрузить версию с сервера или скачать свой черновик. Затем схема проверяется по профилю редактора и XSD. Если у диаграммы есть активный запрос агента, его готовое предложение переносится на новую ревизию, а если перенос невозможен, запрос помечается устаревшим.

4.3 Импорт и выгрузка BPMN-файла

Процесс на рисунке 7 связывает schemix с внешними редакторами.

1) Импортированный файл проверяется по XSD.

2) Схема в профиле редактора приводится к нему и становится редактируемой. Схема вне профиля сохраняется байт в байт и открывается только для чтения.

3) Готовая схема выгружается в BPMN 2.0, SVG, Markdown или DOCX.

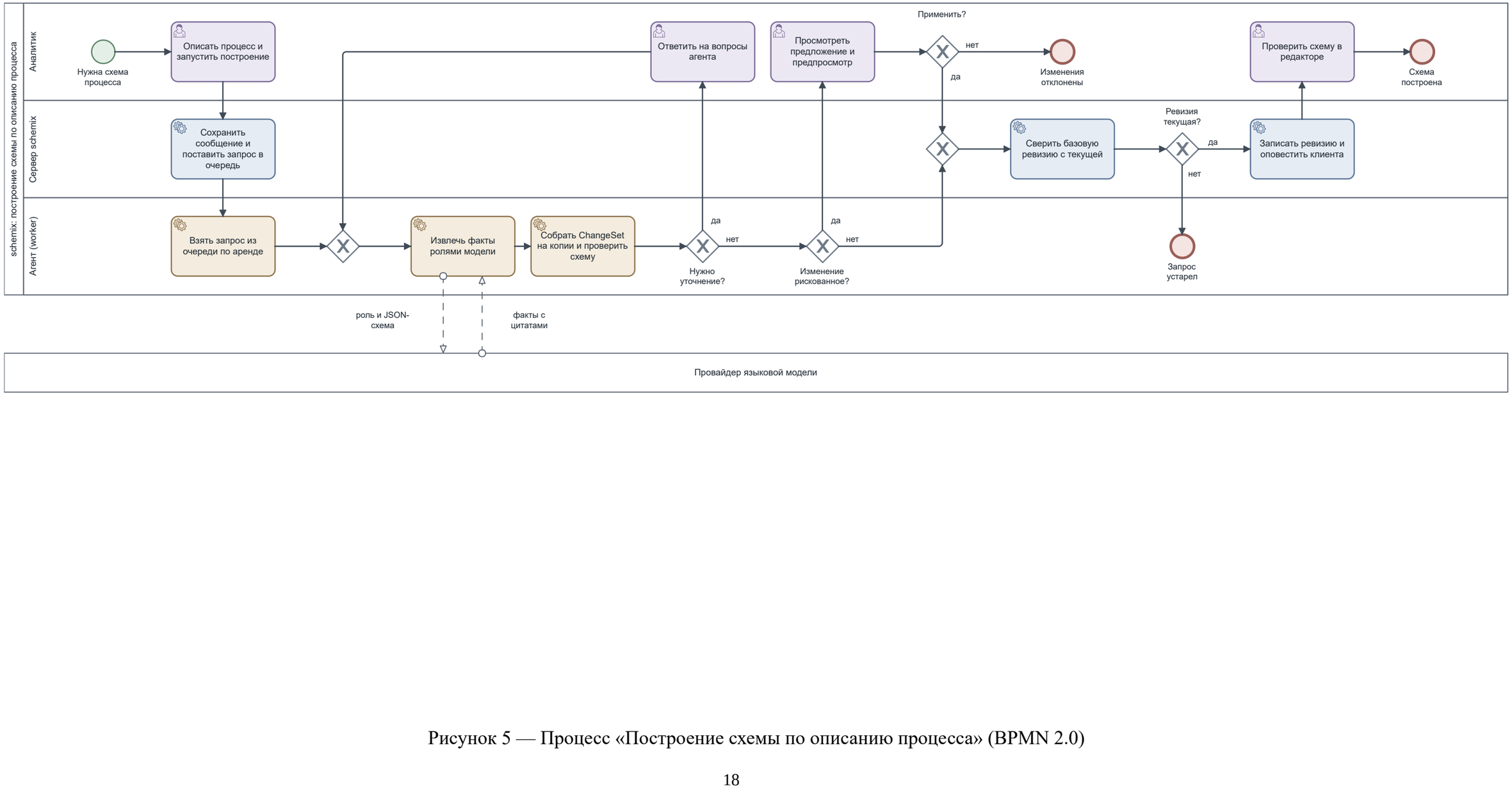


Рисунок 5 — Процесс «Построение схемы по описанию процесса» (BPMN 2.0)

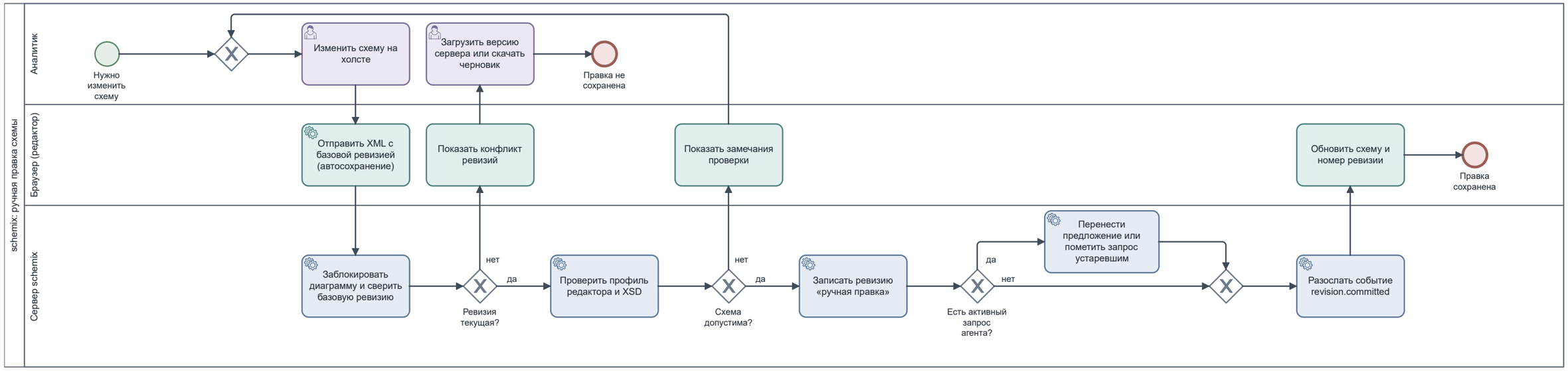


Рисунок 6 — Процесс «Ручная правка схемы» (BPMN 2.0)

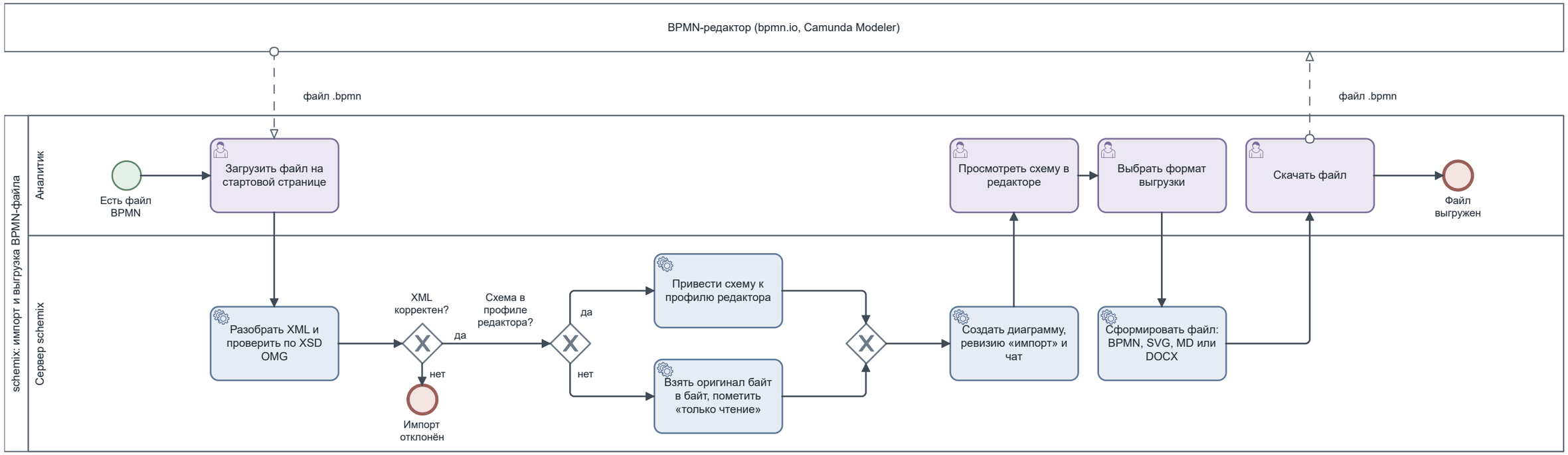


Рисунок 7 — Процесс «Импорт и выгрузка BPMN-файла» (BPMN 2.0)

5 Архитектура

Архитектура описана в модели C4 Саймона Брауна:

– человек — тёмная фигура;

– проектируемая система и её контейнеры — синие прямоугольники;

– внешние системы — серые прямоугольники;

– граница системы — пунктирная рамка;

– зависимость — пунктирная стрелка с назначением и технологией в квадратных скобках.

5.1 Контекст системы

На рисунке 8 показано окружение schemix:

– **аналитик** работает с системой через браузер;

– **провайдер языковой модели** извлекает факты по JSON-схеме;

– **сервис распознавания речи** переводит фразы диктовки в текст;

– **BPMN-редакторы** получают выгруженные файлы — их открывает аналитик.

5.2 Контейнеры

На рисунке 9 система разложена на запускаемые части. Все они разворачиваются одной командой Docker Compose.

– **Веб-приложение** на React 19 и bpmn-js 18 работает в браузере.

– **Веб-сервер nginx 1.28** раздаёт сборку веб-приложения и проксирует `/api` и WebSocket.

– **API-приложение на Python 3.12 и FastAPI** обслуживает HTTP API и поток событий. В нём же работает worker агента.

– **Миграции Alembic** один раз создают схему базы данных.

– **База данных PostgreSQL 16** хранит диаграммы, ревизии, чаты, запросы агенту и журнал событий.

Worker агента и API работают в одном контейнере. Очередью служит таблица запросов в PostgreSQL: worker забирает из неё запрос по аренде.

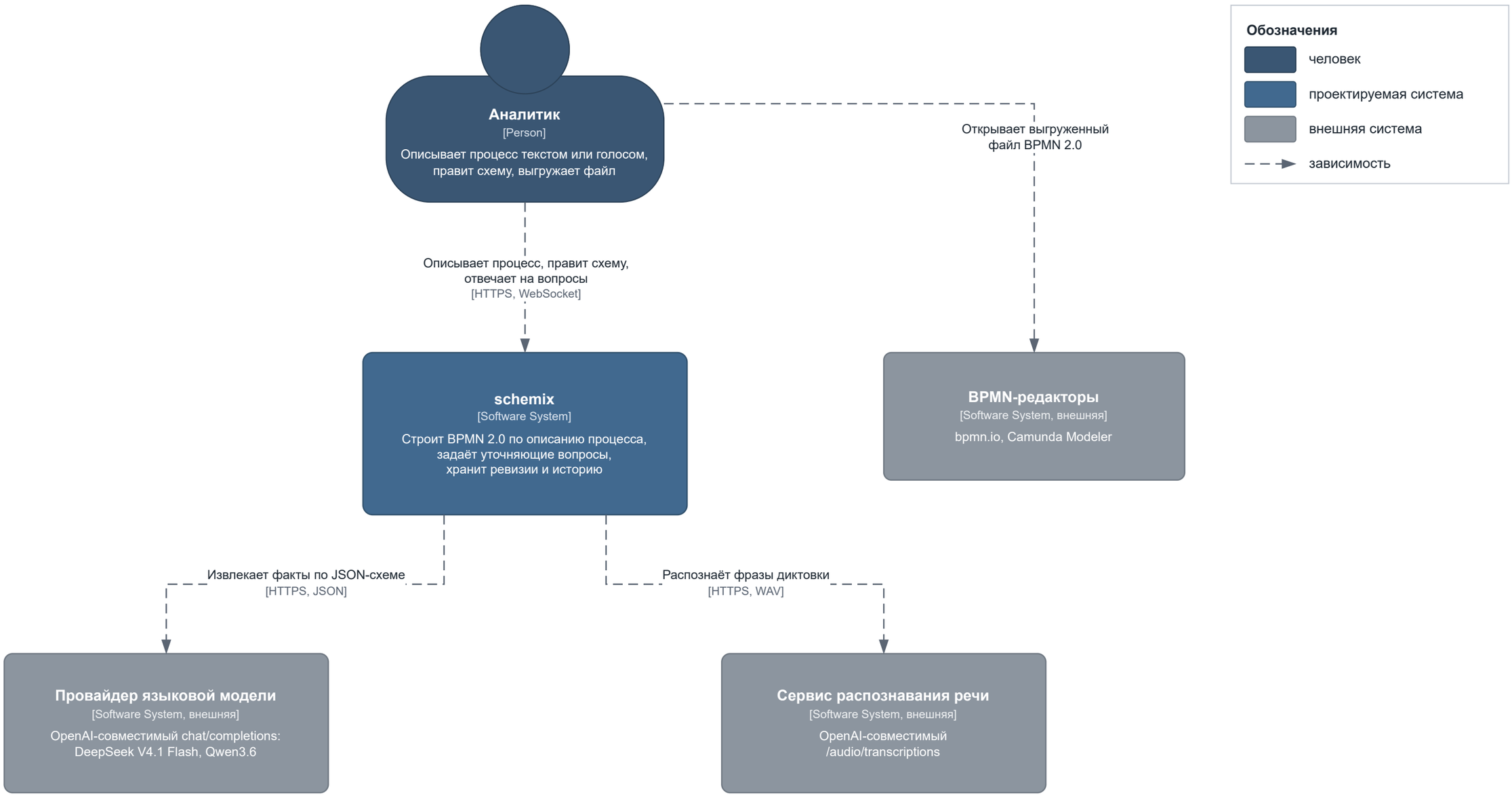


Рисунок 8 — Архитектура schemix: контекст системы (C4, уровень 1)

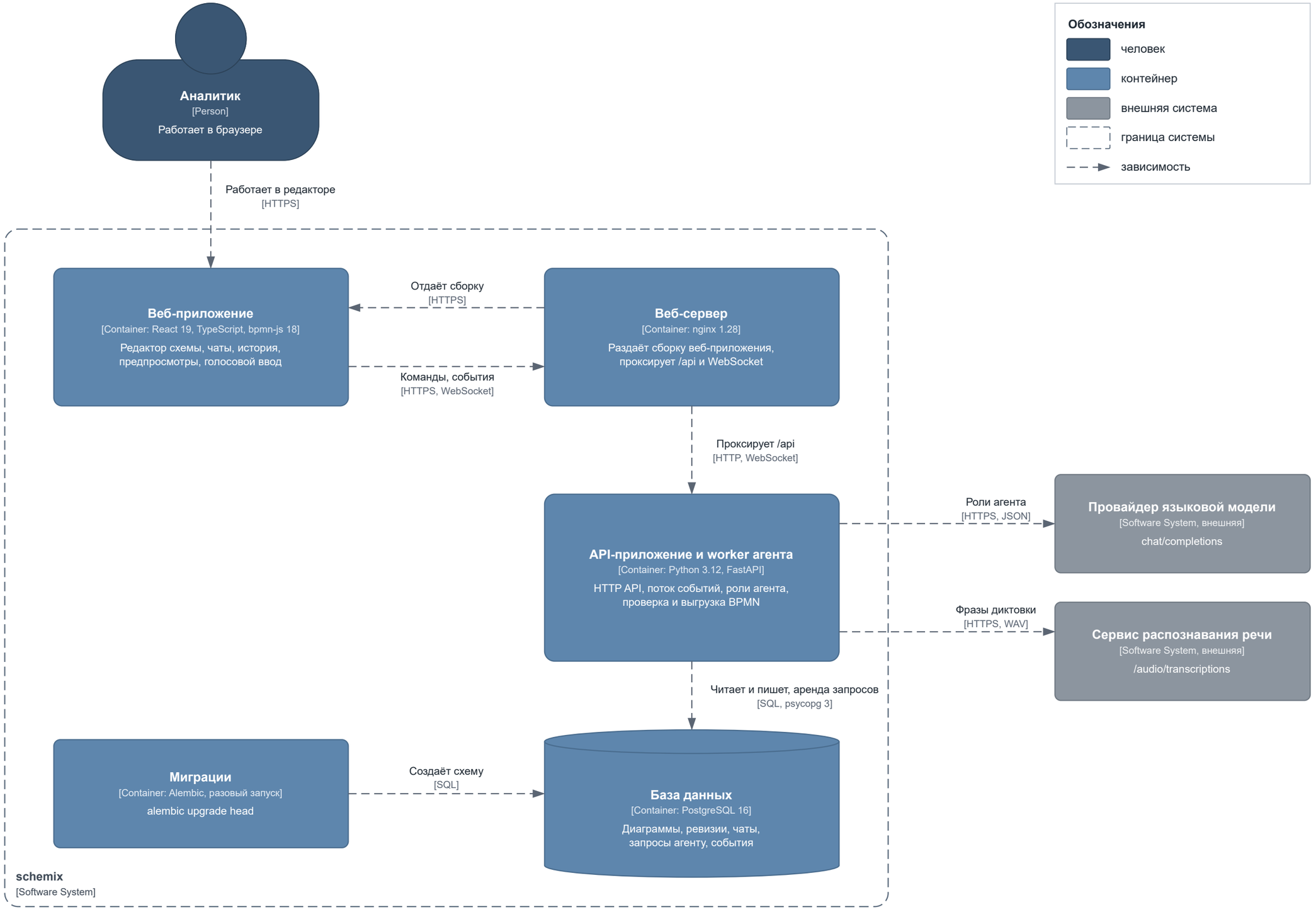


Рисунок 9 — Архитектура schemix: контейнеры (C4, уровень 2)

6 Технический проект программного обеспечения

Технический проект программного обеспечения представлен диаграммой компонентов C4 уровня 3 (рисунок 10) для контейнера «API-приложение и worker агента». Компоненты соответствуют модулям `backend/src/bpmn_assistant`:

- **Входные точки:**
  - HTTP API (`controller/http`) — маршруты `/api/v1` с проверкой идемпотентности;
  - поток событий (`controller/ws`) — WebSocket с прогрессом, черновиком и ревизиями.
- **Сценарии** (`application/workspace`) — создание и импорт диаграмм, запись ревизий с проверкой compare-and-set, чаты, выгрузка.
- **Агент:**
  - Worker берёт запросы по аренде;
  - Runner ведёт запрос по этапам и считает бюджеты;
  - Роли агента (`adapter/agents`) хранят промпты и JSON-схемы ролей.
- **Диктовка** (`application/dictation`) — распознаёт фразы и строит по ним черновик.
- **BPMN-модуль** (`service/bpmn`, `adapter/bpmn`):
  - проверка по XSD и правилам графа;
  - раскладка элементов;
  - нормоконтроль;
  - выгрузка.
- **Адаптеры внешних систем:**
  - клиент модели (порт `LanguageModel`) — OpenAI-совместимый API или Claude Agent SDK;
  - клиент распознавания (порт `SpeechToText`);
  - хранилище (порт `UnitOfWork`) — репозитории и транзакции на psycopg 3.

Все компоненты, которым нужна база данных, работают через одно хранилище, поэтому оно показано общей шиной внизу диаграммы. Компоненты приложения зависят только от портов, адаптеры реализуют эти порты.

Такое разделение (порты и адаптеры) позволяет заменить провайдера модели или базу данных, не меняя сценарии.

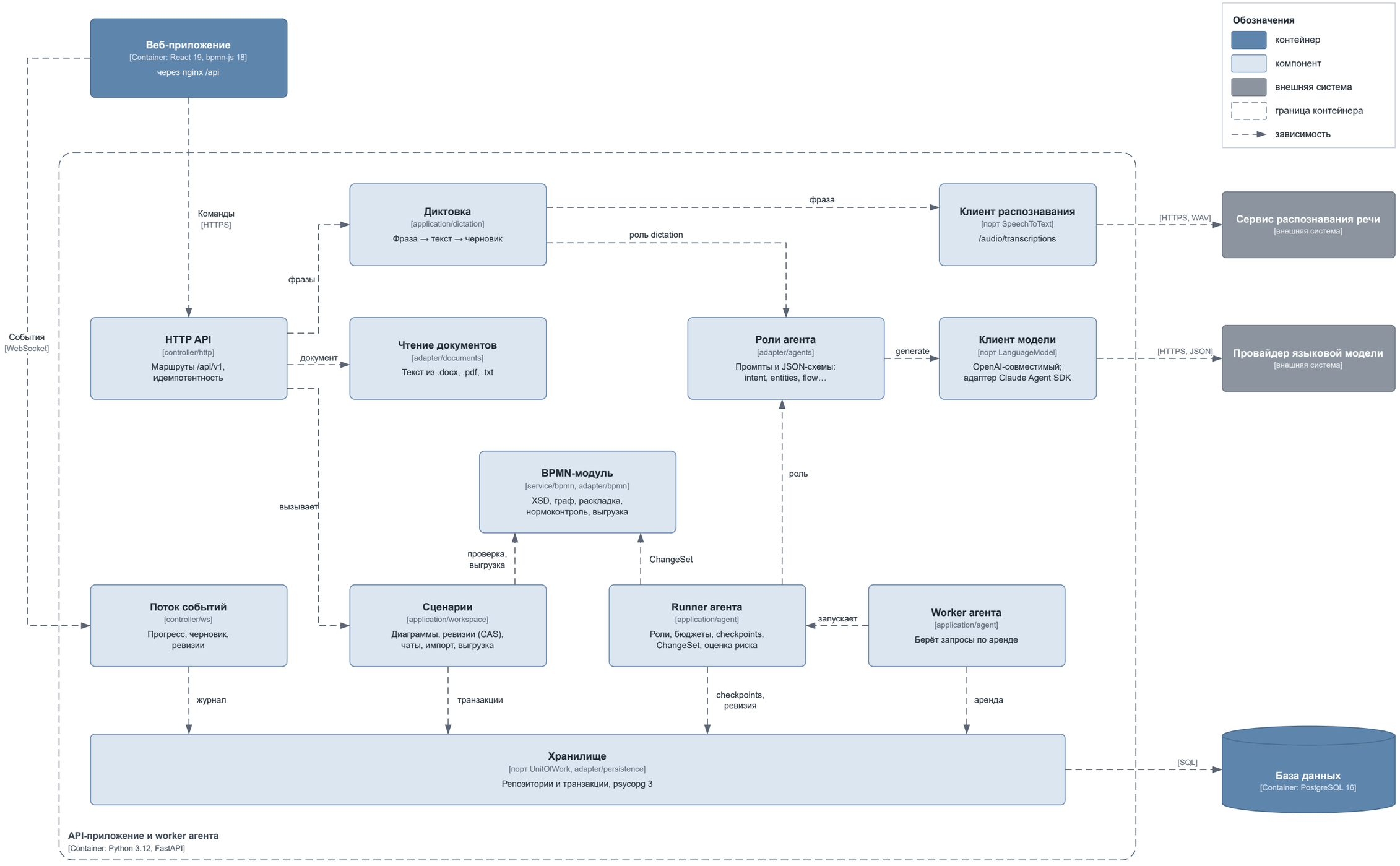


Рисунок 10 — Технический проект ПО: компоненты контейнера «API-приложение и worker агента» (C4, уровень 3)

7 Требования к программному компоненту «Агент»

Структура раздела следует ГОСТ 19.201-78 «Техническое задание. Требования к содержанию и оформлению». Требования относятся к компоненту «Агент» — части контейнера «API-приложение и worker агента» (рисунок 10), которая превращает сообщение аналитика в изменение схемы. Числовые значения взяты из кода schemix: файлы `limits.py` и `main.py` пакета `bpmn_assistant`.

7.1 Назначение

Компонент принимает запрос из очереди и строит по тексту сообщения изменение схемы BPMN 2.0. Он задаёт аналитику уточняющие вопросы, если в тексте не хватает фактов. Рискованное изменение компонент передаёт на подтверждение с предпросмотром, а безопасное записывает новой ревизией.

Состав компонента показан на UML-диаграмме компонентов (рисунок 12):

– **Worker агента** (`application/agent/worker`) берёт запросы из очереди по аренде;

– **Runner агента** (`application/agent/runner`) ведёт запрос по этапам;

– **Роли агента** (`adapter/agents`) — промпты и JSON-схемы ролей модели;

– **Клиент модели** (`adapter/llm`) — обращения к провайдеру языковой модели.

7.2 Функциональные требования

Варианты использования системы показаны на UML-диаграмме вариантов использования (рисунок 11). Компонент «Агент» реализует варианты «Построить схему по описанию», «Ответить на уточняющие вопросы», «Подтвердить или отклонить рискованное изменение» и включаемый вариант «Извлечь факты ролями модели».

Таблица 1 — Функциональные требования к компоненту «Агент»

№	Требование
Ф-1	Брать из очереди запрос в статусе `queued` с арендой на 90 с и продлевать её каждые 15 с. Если аренда истекла, запрос должен подхватить другой воркер

№	Требование
Ф-2	Обрабатывать запрос конвейером ролей `intent → entities → entity_match → flow → relation_match → review`. Роль `clarify` формирует вопросы, если `review` или код нашли неоднозначность
Ф-3	Принимать от модели только факты с цитатами по JSON-схеме роли. XML BPMN строит код по набору изменений (ChangeSet)
Ф-4	Проверять цитаты по исходному тексту. Ответ роли, не прошедший схему или проверку цитат, отправлять на исправление владельцу этапа, не больше 2 циклов на этап
Ф-5	Применять ChangeSet к копии схемы, раскладывать элементы и проверять кандидата по XSD OMG BPMN 2.0 и правилам графа
Ф-6	Если фактов не хватает, задавать вопросы с ранжированными вариантами ответа: не больше 3 раундов, не больше 3 вариантов на вопрос
Ф-7	Классифицировать риск кодом по ChangeSet. Рискованное изменение публиковать как предложение с предпросмотром, безопасное — как новую ревизию
Ф-8	Записывать ревизию через compare-and-set по базовой ревизии. Если база устарела, переводить запрос в статус `stale`
Ф-9	Сохранять ответ каждой роли (артефакт) с версиями промпта, модели и JSON-схемы. После ответа на вопрос пересчитывать только затронутые этапы
Ф-10	Публиковать в поток событий диаграммы ход работы, черновик и итог запроса

7.3 Интерфейсы компонента

Предоставляемые и требуемые интерфейсы показаны на рисунке 12.

Таблица 2 — Интерфейсы компонента «Агент»

Интерфейс	Вид	Операции	Кто реализует
`RunProcessor`	предоставляемый	`process(run_id, lease_token)`	Runner агента
`StageAgent`	требуемый	`execute(context, input) → StageResult`	Роли агента
`LanguageModel`	требуемый (ролями)	`generate(ModelRequest) → ModelResponse`	Клиент модели: OpenAI-совместимый API или Claude Agent SDK
`BpmnService`	требуемый	`inspect`, `prepare_changes`, `validate`	BPMN-модуль
`RunCompletion`	требуемый	`complete_text`, `publish_questions`, `publish_candidate`, `fail`	Сценарии (`application/workspace`)
`UnitOfWork`	требуемый	репозитории, `commit`, `rollback`	Хранилище (PostgreSQL)
`StreamEvents`	требуемый	`publish(repository, event)`	Поток событий
`Clock`, `IdGenerator`	требуемые	`now()`, `new_id()`	системные адаптеры

Компонент зависит только от портов `application/ports`. Направление импортов (адаптеры → приложение → модели) проверяет архитектурный тест `tests/test_architecture.py`.

7.4 Требования к надёжности

– Запрос, не завершённый за 24 ч, истекает. Запрос с истекшей арендой возвращается в очередь (`recover_expired`).

– Вызов модели повторяется не больше 2 раз при кодах 429, 500, 502, 503, 504.

– У диаграммы не больше одного активного запроса (частичный уникальный индекс в базе).

– Повторная доставка команды с тем же `Idempotency-Key` не создаёт второго запроса.

– После перезапуска сервиса запрос не теряется. Это проверено E2E-тестом `production-restart.spec.ts`.

7.5 Ограничения ресурсов

Таблица 3 — Ограничения ресурсов компонента «Агент»

Параметр	Значение по умолчанию	Переменная окружения
Время одного вызова модели	60 с	—
Бюджет активной работы раунда	180 с	—
Вызовов модели на запрос	40	`RUN_MAX_MODEL_CALLS`
Токенов на запрос	160 000	`RUN_MAX_TOKENS`
Токенов на вход этапа / выход	32 000 / 4 000	`RUN_MAX_INPUT_TOKENS`
Веток предпросмотра	9	`RUN_MAX_PREVIEW_BRANCHES`
Одновременных обращений к провайдеру	5	—
Стоимость запроса	без ограничения	`RUN_MAX_COST_RUB`
Длина сообщения	20 000 символов	—
Размер XML схемы	5 МиБ	—

7.6 Условия эксплуатации и технические средства

– Запуск: Docker Compose. Контейнеры: PostgreSQL 16, миграции Alembic, backend (Python 3.12, FastAPI), frontend (nginx 1.28).

– Память backend в покое — около 240 МБ. Пилотная ВМ стоит 6 662 ₽/мес, демонстрационная — 3 426 ₽/мес (по `economics.md`).

– Нужен доступ к OpenAI-совместимому провайдеру модели. Ключ хранится только в окружении backend. Без ключа работают редактор, импорт и выгрузка, агент недоступен.

7.7 Информационная и программная совместимость

– Результат — XML BPMN 2.0, проходящий XSD OMG; файлы открываются в bpmn.io и Camunda Modeler.

– Провайдер модели — любой с OpenAI-совместимым `chat/completions` и строгим JSON-выводом. Проверены DeepSeek V4.1 Flash и Qwen3.6 35B.

– Распознавание речи — OpenAI-совместимый `/audio/transcriptions`, WAV 16 кГц моно.

– XML разбирается без внешних сущностей, DTD и сетевого доступа (защита от XXE).

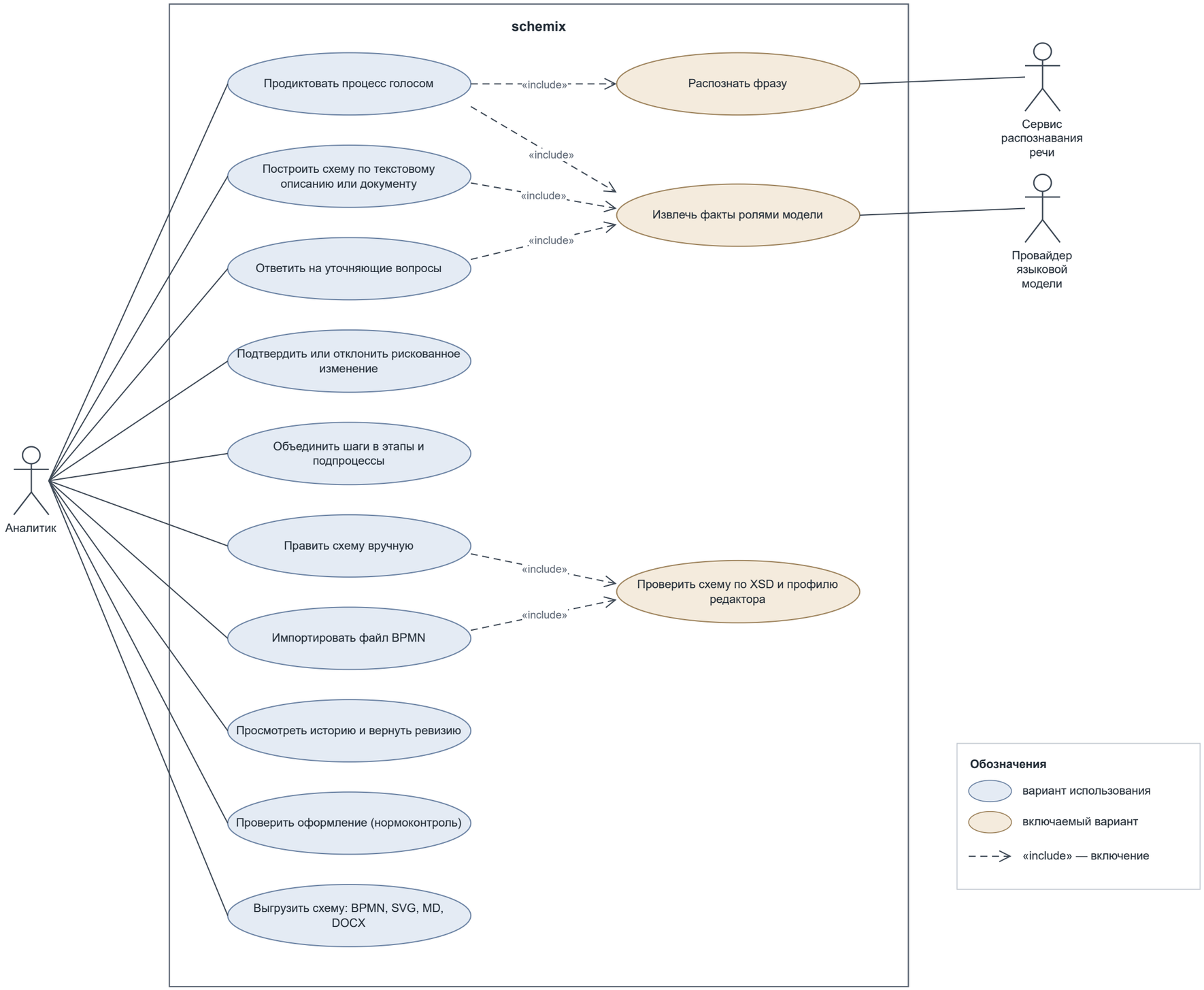


Рисунок 11 — UML-диаграмма вариантов использования schemix

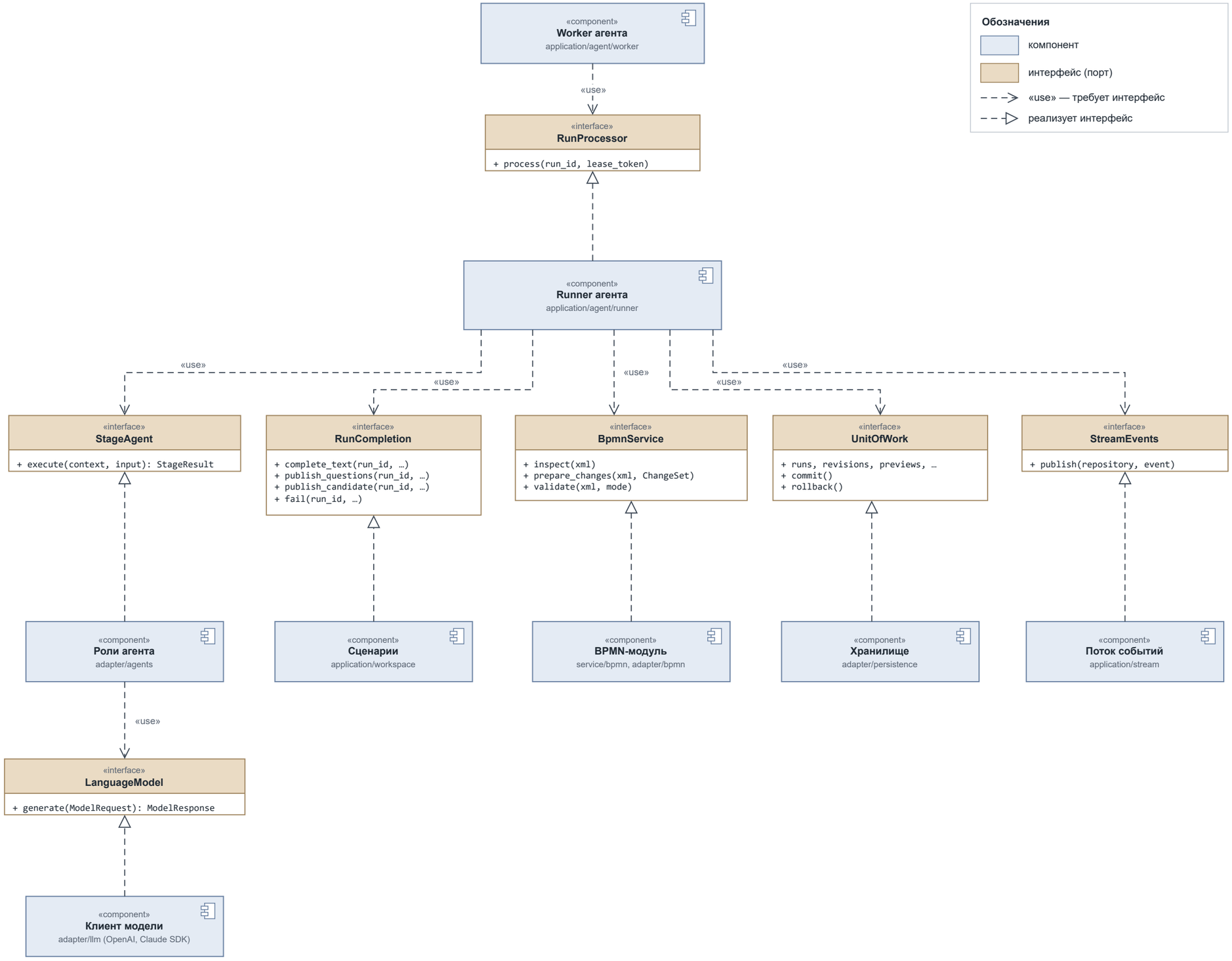


Рисунок 12 — UML-диаграмма компонентов агента: предоставляемые и требуемые интерфейсы

8 Оценка качества

8.1 Методика

Качество оценено по модели качества продукта ГОСТ Р ИСО/МЭК 25010-2015 «Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов». Модель включает восемь характеристик:

– функциональная пригодность;
– уровень производительности;
– совместимость;
– удобство использования;
– надёжность;
– защищённость;
– сопровождаемость;
– переносимость.

Для каждой характеристики выбраны измеримые показатели, а значения взяты из артефактов проекта schemix: протоколов тестов, отчётов прогонов агента, замеров производительности. Для каждого значения указан источник в репозитории `interrao-hack`. Итог по характеристике выставлен по трёхуровневой шкале:

– **соответствует** — показатели достигнуты и подтверждены измерением;
– **частично** — часть показателей не достигнута или не измерялась;
– **не соответствует** — требование не выполнено.

8.2 Оценка по характеристикам

Таблица 4 — Оценка качества schemix по характеристикам ГОСТ Р ИСО/МЭК 25010-2015

Характеристика	Показатель	Значение (источник)	Итог
Функциональная пригодность	Доля прогонов, где схема получена и применена (50 параллельных запросов)	24 из 50 применены, 12 ждут подтверждения, 3 ждут ответа, 8 упали из-за недоступности провайдера, 3 без схемы (`tasks/reports/AI-10-50-runs.md`)	частично

Характеристика	Показатель	Значение (источник)	Итог
	Применены по размеру процесса	6 действий — 10/10; 12 действий — 3/10; 24 действия — 2/10; эталонные — 9/20 (там же)	
	Полнота извлечения: короткие процессы / длинные регламенты	77 % пунктов (96 из 125), схема в 19 из 20 / 43 % (95 из 223), схема в 17 из 20 (`tasks/reports/e2e-2026-10-22/README.md`)	
	Корректность BPMN	все сохранённые ревизии проходят XSD OMG; 7 из 7 примеров открываются в demo.bpmn.io (`README.md`, `docs/evidence.md`)	
Уровень производительности	Время ответа API	проверка схемы 3,7 мс (168 запросов/с на ядро), нормоконтроль 4,2 мс, выгрузка SVG 6,6 мс (`economics.md`)	частично
	Время построения схемы	короткий процесс: 33–49 с у DeepSeek, 169 с у Qwen3.6; медиана 45 с на наборе из 20 коротких процессов (`AI-10-model-comparison.md`, отчёт e2e)	
	Работа под нагрузкой	при 50 одновременных запросах медиана 21,6 мин; запуск в работу — 4,7 с; нагрузочное тестирование не проводилось (`AI-10-50-runs.md`, `docs/limitations.md`)	
	Затраты ресурсов	память backend около 240 МБ; одна схема — 13,9–128 ₽ (типовая 22,9 ₽) (`economics.md`)	
Совместимость	Обмен файлами	BPMN 2.0 по XSD OMG, импорт и выгрузка; открывается в bpmn.io и Camunda Modeler; выгрузка также в SVG, Markdown, DOCX (код экспорта, `docs/evidence.md`)	соответствует
	Сосуществование	работа с любым OpenAI-совместимым провайдером; совместимость с BPMN-системой заказчика не проверялась (`docs/agent.md`, `docs/evidence.md`)	
Удобство использования	Понятность	интерфейс на русском; вопросы агента с вариантами «Рекомендуемый» и «Не знаю» и предпросмотром каждого варианта (`frontend/src/shared/i18n`, `docs/agent.md`)	частично

Характеристика	Показатель	Значение (источник)	Итог
	Защита от ошибок пользователя	рискованные изменения ждут подтверждения; конфликт ревизий показывается с возможностью скачать черновик (`docs/agent.md`, ADR 0002)	
	Доступность	E2E и горячие клавиши проверены только в Chromium; микрофон работает только по HTTPS; юзабилити-тестирование с пользователями не проводилось (`docs/limitations.md`)	
Надёжность	Отказоустойчивость	аренда запроса с heartbeat, возврат истёкших запросов; перезапуск сервиса без потери запроса проверен E2E (`worker.py`, `docs/evidence.md`)	частично
	Целостность данных	compare-and-set ревизий, отложенные внешние ключи, идемпотентность команд (ADR 0002–0003, миграции)	
	Доступность внешнего сервиса	8 из 50 прогонов упали с ошибкой «provider is unreachable»; резервного провайдера нет (`AI-10-50-runs.md`)	
Защищённость	Конфиденциальность и целостность	разбор XML без внешних сущностей, DTD и сети; контрольные суммы XSD; ключ модели только в окружении backend; проверка Origin у WebSocket (`xmlio.py`, `main.py`)	частично
	Подотчётность и разграничение доступа	входа и прав нет, у всех пользователей одно пространство (`docs/limitations.md`)	
Сопровождаемость	Модульность	слои controller / application / ports / adapter / service / models; 215 файлов Python, около 48,7 тыс. строк; направление импортов проверяет тест (`backend/src`, `tests/test_architecture.py`)	соответствует
	Тестируемость	бэкенд: 873 теста пройдено, 3 пропущено; фронтенд: 397 unit-тестов; 16 сценариев Playwright E2E; ruff, ESLint, tsc без замечаний (`docs/running.md` (прогон 13.10.2026))	

Характеристика	Показатель	Значение (источник)	Итог
	Анализируемость	4 ADR; ответы ролей сохраняются с версиями промпта и модели; покрытие кода не измеряется, CI нет (`docs/adr`, `docs/limitations.md`)	
Переносимость	Устанавливаемость	один `docker compose up`: PostgreSQL 16, миграции, backend, nginx (`docker-compose.yml`)	соответствует
	Адаптируемость	провайдер модели и распознавания задаётся переменными окружения; модели Яндекса не проверены (`docs/agent.md`, `docs/limitations.md`)	

8.3 Качество результата работы агента

Кроме характеристик продукта оценено качество схем, которые строит агент. Методика серии AI-10 (23.10.2026, версия `ae0a861`) такая:

– 50 уникальных описаний запущены одновременно;

– проверены статусы, геометрия сохранённого XML и соответствие исходному тексту.

Результаты:

– **Геометрия.** В сохранённых схемах нет пересечений связей, наложений фигур, выходов из дорожек и вырожденных шлюзов. Две связи данных проходят поверх фигуры.

– **Соответствие тексту** (предварительная оценка):

  – 24 схемы соответствуют описанию;

  – у 11 есть смысловые замечания, например проверка документов поставлена до этапа, а не после;

  – для 14 запросов схемы нет.

– **Крайние случаи** (10 сценариев, модель DeepSeek V4.1 Flash): схема применена в 4 случаях, 1 ждёт подтверждения, в 5 схемы нет. Все проверки оракула прошла 1 схема из 10.

Сам проект прямо указывает, что доля успеха и точность извлечения на наборе с эталонными схемами пока не измерены (`docs/limitations.md`).

8.4 Выводы

Сильные стороны.

– Корректность выходного формата гарантирована проверкой по XSD OMG.

– Целостность данных обеспечена compare-and-set и отложенными ключами.

– Модульная архитектура с портами и адаптерами.

– Большой автоматический набор тестов.

– Короткие процессы (до 6 действий) строятся надёжно: 10 из 10 применены.

Слабые стороны.

– Длинные регламенты обрабатываются плохо: применены 2 из 10 схем на 24 действия, полнота извлечения 43 %.

– Система зависит от одного провайдера модели: 16 % прогонов упали из-за его недоступности.

– Нет аутентификации и разграничения прав.

– Нет CI и измерения покрытия тестами.

– Нагрузочное тестирование не проводилось.

Рекомендации.

1) Разбивать длинный регламент на этапы до запуска ролей, чтобы повысить долю применённых схем.

2) Добавить резервного провайдера модели и повтор запроса при недоступности основного.

3) Ввести вход по учётной записи организации и права на диаграммы.

4) Включить прогон тестов и измерение покрытия в CI.

5) Провести нагрузочное тестирование на целевое число пользователей.

Итог: из восьми характеристик ГОСТ Р ИСО/МЭК 25010 три соответствуют полностью (совместимость, сопровождаемость, переносимость), пять — частично. Не соответствующих характеристик нет. Система пригодна для пилотной эксплуатации на коротких и средних процессах. Для промышленной эксплуатации нужны аутентификация, отказоустойчивость к сбоям провайдера и повышение качества на длинных регламентах.